

EvalForge Results and Parameters Report

Backend Service for Generating and Running Evaluation Benchmarks for RAG Systems and AI-Agent Workflows

Generated from dashboard screenshots, exported JSON/JSONL/CSV artifacts, terminal validation output, and Docker execution logs.

1. Executive Summary

EvalForge was successfully validated as an end-to-end evaluation infrastructure project. The tested workflow supports file upload, document ingestion, chunking, rule extraction, benchmark-case generation, citation validation, optional evaluation, SQLite persistence, export generation, run comparison, Dockerized execution, and a browser dashboard. The main dashboard run completed successfully with 5 uploaded files, 20 generated benchmark cases, perfect validation/citation coverage, and a demo evaluation score around 0.855.

Category	Result
Project Status	Working MVP / portfolio-grade backend and dashboard
Primary Interface	FastAPI dashboard at /dashboard plus Swagger at /docs
Main Endpoint	POST /pipeline/run
Persistence	SQLite-backed saved pipeline runs
Exports Verified	JSON, JSONL, CSV, quality report
Docker Status	Docker images built and containers started successfully
Final Test Status	123 passed, 757 warnings in 4.52 seconds

2. Implemented Capabilities

Capability	Evidence / Description
Document ingestion	Supports .md, .txt, .json, .csv, and .pdf files.
CSV handling	Large CSV ingestion capped/sampled by default to keep generated chunks/rules manageable.
Pipeline automation	One click/API call performs loading, chunking, rule extraction, test generation, validation, persistence, and optional evaluation.
Test generation	Grounded QA, ambiguity/clarification, adversarial/policy-bypass, and tool-use correctness.
Validation	Checks schema validity, citation coverage, risk labels, review status, and warning/invalid case summaries.
Evaluation	Demo target evaluation and HTTP target adapter for external RAG/agent APIs.
Persistence	Pipeline runs, cases, quality summaries, and evaluation summaries are saved in SQLite.
Exports	Persisted runs export as JSON, JSONL, CSV, and quality report.
Run comparison	Compares current and baseline pipeline runs to identify improvements/regressions.
Deployment	Docker Compose starts FastAPI backend and Streamlit application.

3. Main Dashboard Run: Parameters and Results

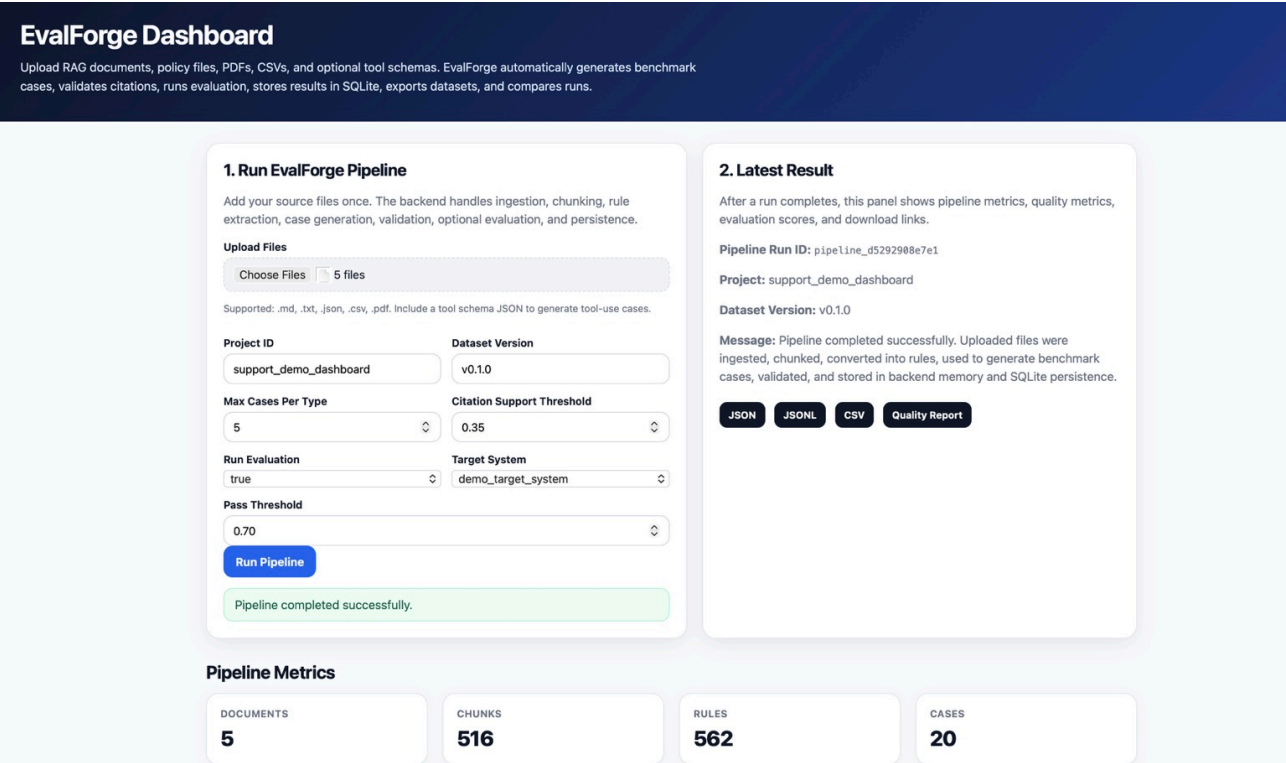
The dashboard interface was tested with five uploaded files: refund_policy.md, shipping_policy.md, warranty_policy.md, support_tool_schema.json, and Customer_support_data.csv. After adding the CSV ingestion cap, the run became much more stable and produced manageable counts.

Parameter	Value
Project ID	support_demo_dashboard
Dataset Version	v0.1.0
Uploaded Files	5
Run Evaluation	true
Target System	demo_target_system
Max Cases Per Type	5
Citation Support Threshold	0.35
Pass Threshold	0.70

3.1 Dashboard Metrics Achieved

Metric	Value
Documents	5
Chunks	516
Rules	562
Generated Cases	20
Validity Rate	1.000
Citation Coverage	1.000
Pass Rate	1.000
Average Score	0.855
Test Type Distribution	grounded_policy_qa: 5; ambiguity: 5; adversarial: 5; tool_use_correctness: 5
Risk Distribution	low: 4; medium: 11; high: 5

3.2 Dashboard Evidence



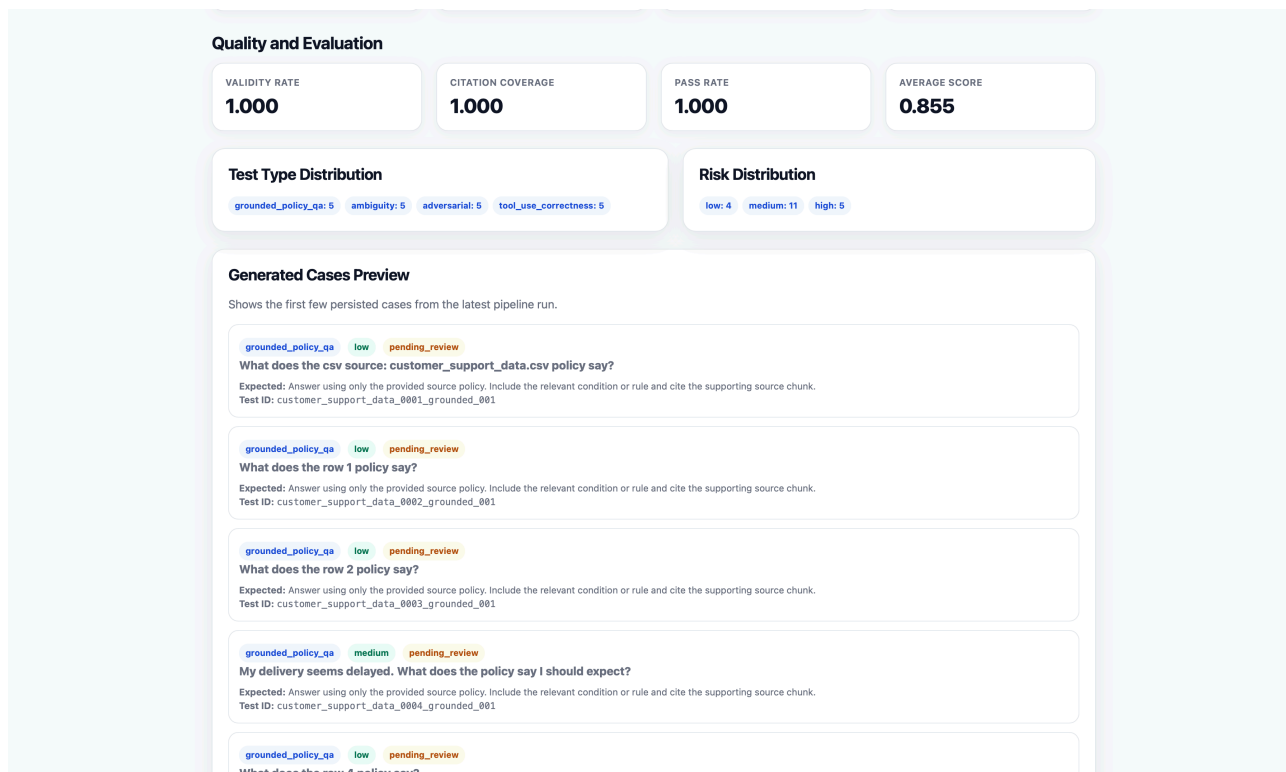


Figure 2. Dashboard-generated case preview showing generated benchmark cases and test-type labels.

These are saved in SQLite. You can open exports, inspect cases, or compare two runs.

Refresh Runs

RUN ID	PROJECT	VERSION	CASES	VALIDITY	SCORE	EXPORTS
pipeline_d5292988e7e1	support_demo_dashboard	v0.1.0	20	1.000	0.855	JSON CSV Cases
pipeline_8c4e05926872	support_demo_dashboard	v0.1.0	20	1.000	0.855	JSON CSV Cases
pipeline_120818389ac6	support_demo_dashboard	v0.1.0	20	1.000	0.855	JSON CSV Cases
pipeline_52aea757c970	support_demo_dashboard	v0.1.0	20	1.000	0.855	JSON CSV Cases
pipeline_b46fcd7f6210	support_demo_comparison	v0.1.0	16	1.000	0.427	JSON CSV Cases
pipeline_19f5b549e07b	support_demo_comparison	v0.1.0	16	1.000	0.860	JSON CSV Cases
pipeline_f0614a20737f	support_demo_comparison	v0.1.0	16	1.000	0.860	JSON CSV Cases
pipeline_0781f0772864	support_demo_comparison	v0.1.0	16	1.000	0.427	JSON CSV Cases
pipeline_3c68b982c981	support_demo	v0.1.0	16	1.000	0.427	JSON CSV Cases
pipeline_eca44b408c78	support_demo	v0.1.0	16	1.000	0.860	JSON CSV Cases
pipeline_9b558afaa80b	support_demo	v0.1.0	16	1.000	0.860	JSON CSV Cases
pipeline_3d830944ceb9	support_demo	v0.1.0	16	1.000	—	JSON CSV Cases
pipeline_d9903bec4e04	support_demo_persistence	v0.1.0	12	1.000	0.858	JSON CSV Cases
pipeline_95be2ba96437	support_demo_exports	v0.1.0	12	1.000	0.858	JSON CSV Cases
pipeline_ea723a7ba2eb	support_demo_exports	v0.1.0	12	1.000	0.858	JSON CSV Cases
pipeline_f7dc42b146fa	support_demo_exports	v0.1.0	12	1.000	0.858	JSON CSV Cases
pipeline_71c3e4cf16da	support_demo_exports	v0.1.0	12	1.000	0.858	JSON CSV Cases
pipeline_832fc8ca0cd4	support_demo_exports	v0.1.0	12	1.000	0.858	JSON CSV Cases
pipeline_97f3ec47a834	support_demo_exports	v0.1.0	12	1.000	0.858	JSON CSV Cases
pipeline_85b9f9b789a1	support_demo_exports	v0.1.0	12	1.000	0.858	JSON CSV Cases

Figure 3. Persisted run table showing saved SQLite runs, validity scores, average scores, and export links.

4. Exported Dataset Artifact Results

The uploaded JSON/JSONL/CSV artifacts were inspected to validate that the generated benchmark dataset contains structured cases and complete metadata.

Field	Value
Exported Run ID	pipeline_c1c90f121de1
Project ID	support_demo_interface
Dataset Version	v0.1.0
Documents	4
Chunks	11
Rules	16
Cases	20
Created At	2026-05-17T17:10:33.954070

4.1 Quality Summary From Export

Metric	Value
Total Cases	20
Valid Cases	20
Invalid Cases	0
Validity Rate	100.0%
Cases With Citations	20
Citation Coverage	100.0%
Total Errors	0
Total Warnings	20
Review Status	pending_review: 20

4.2 Evaluation Summary From Export

Metric	Value
Eval Run ID	evalrun_3d4448fde2
Target System	demo_target_system
Total Cases	20
Passed Cases	20
Failed Cases	0
Pass Rate	100.0%
Average Score	0.8603
Pass Threshold	0.7
Test Type	Score
grounded_policy_qa	0.9683
ambiguity	0.8332
adversarial	0.8046
tool_use_correctness	0.8352

4.3 Export Format Verification

Format	File	Observed Content	Status
CSV	pipeline_e7b19e507b5c.csv	20 rows x 16 columns	Readable and structured
JSONL	pipeline_e7b19e507b5c.jsonl	20 JSON lines	Readable line-delimited dataset
JSON	response_1779038081717.json	metadata + cases	Readable benchmark bundle

5. Generated Benchmark Case Breakdown

Test Type	Count
grounded_policy_qa	5
ambiguity	5
adversarial	5
tool_use_correctness	5

Risk Level	Count
medium	11
high	5
low	4

Representative generated cases from the exported CSV:

Test ID	Type	Risk	User Query
customer_support_data_0002_grounded_001	grounded_policy_qa	low	What does the row 1 policy say?
customer_support_data_0003_grounded_001	grounded_policy_qa	low	What does the row 2 policy say?
customer_support_data_0004_grounded_001	grounded_policy_qa	medium	My delivery seems delayed. What does the policy say I should expect?
customer_support_data_0005_grounded_001	grounded_policy_qa	low	What does the row 4 policy say?
customer_support_data_0006_grounded_001	grounded_policy_qa	low	What does the row 5 policy say?
customer_support_data_0004_ambiguity_001	ambiguity	medium	I need help with row 3. What should I do?
customer_support_data_0009_ambiguity_001	ambiguity	medium	I need help with row 8. What should I do?
customer_support_data_0011_ambiguity_001	ambiguity	medium	I need help with row 10. What should I do?

6. Policy-Only API Run Results

A policy-only run using refund_policy.md, shipping_policy.md, warranty_policy.md, and support_tool_schema.json produced a clean compact dataset. This is the recommended demo path for showing high-quality policy-grounded generation.

Metric	Value
Documents	4
Chunks	11
Rules	16
Generated Cases	20
Validity Rate	1.0
Citation Coverage	1.0
Test Types	5 grounded QA, 5 ambiguity, 5 adversarial, 5 tool-use correctness
Risk Distribution	14 medium, 6 high
Demo Eval Average Score	0.8603
Demo Eval Pass Rate	1.0

7. HTTP Target Evaluation Results

EvalForge was also tested against a real HTTP target adapter using the fake RAG target service. This proves the system can evaluate an external RAG/agent API, not just a hardcoded demo target.

Metric	Value
Target URL	http://127.0.0.1:9000/answer
Target System Name	http_target:http://127.0.0.1:9000/answer
Total Cases	20
Passed Cases	12
Failed Cases	8
Pass Rate	0.600
Average Score	0.7202
Grounded QA Score	0.8010
Ambiguity Score	0.6972
Adversarial Score	0.7613
Tool-Use Correctness Score	0.6215

8. Test Suite and Quality Gate Results

The project was repeatedly validated through unit, integration, backend API, persistence, export, dashboard, and HTTP target adapter tests. Warnings were observed, but they are primarily deprecation/collection warnings and did not cause failures.

Test Area	Observed Result
Full test suite	123 passed, 757 warnings in 4.52 seconds
Dashboard test included	Yes
Document loader tests	9 passed, 6 warnings
Pipeline API tests	4 passed, 169 warnings
Persistence tests	1 passed, 36 warnings
Persisted export tests	5 passed, 138 warnings
Run comparison tests	3 passed, 170 warnings
FastAPI backend tests	16 passed, 132 warnings
HTTP target adapter tests	4 passed, 2 warnings

Main warning categories observed:

Warning	Interpretation
PytestCollectionWarning for TestType enum	Non-blocking; pytest sees an enum class name beginning with Test and warns it is not a test case.
FastAPI on_event deprecation	Non-blocking; can be modernized later with lifespan event handlers.
datetime.utcnow deprecation	Non-blocking; can be replaced with datetime.now(datetime.UTC) later.
SQLAlchemy datetime default warning	Non-blocking; related to timezone-aware datetime migration.

9. Docker and Deployment Verification

Docker Desktop was verified, hello-world ran successfully, images were built, and Docker Compose started both EvalForge services. The user manually stopped the services with Ctrl+C after startup, which is expected behavior and not a crash.

Deployment Check	Result
Docker Version	Docker Engine 29.4.0 client; Docker Desktop 4.71.0 server
Architecture	darwin/arm64 client, linux/arm64 Docker server
hello-world	Completed successfully
Docker Compose Build	evalforge-api and evalforge-streamlit images built successfully
Backend Container	Started Uvicorn on 0.0.0.0:8000
Streamlit Container	Started on 0.0.0.0:8501
Health Check	{"status":"ok","app":"EvalForge API","version":"0.2.0","environment":"local"}

10. GitHub Commit Milestones

Commit	Feature / Milestone
04c6f55	Added browser upload UI for EvalForge pipeline.
2fc460f	Added backend-connected dashboard and dashboard test.
0ed1b65	Limited CSV ingestion rows for stable benchmark generation.
9124c10	Added Dockerfiles and docker-compose setup.
9a628bf	Added polished README.
4b8bc9b	Added HTTP target evaluation adapter, fake RAG target, and related tests.

11. Final Interpretation

EvalForge is functioning as a complete evaluation-workflow MVP. The best demo path is to run the dashboard with the four clean policy/tool-schema files first, because those produce high-quality policy-specific benchmark cases. Large

external CSVs are now supported more safely after the ingestion cap, but CSV-only generation may still create row-oriented cases if the source file is operational ticket data rather than a policy document.

Area	Recommendation
Strongest current demo	Upload refund_policy.md, shipping_policy.md, warranty_policy.md, and support_tool_schema.json through /dashboard.
External dataset demo	Upload Customer_support_data.csv with sample policies to show CSV ingestion and realistic support-ticket language.
Backend proof	Use /pipeline/run, /pipeline/runs, /export/json, /export/jsonl, /export/csv, /quality-report, and /compare endpoints.
Evaluation proof	Use demo_target_system for controlled scoring and fake RAG HTTP target for external API evaluation.
Deployment proof	Use docker compose up --build to start API and Streamlit services.

12. Appendix: Key Commands Used

```
python -m pytest -q
# Final observed result: 123 passed, 757 warnings in 4.52s
uvicorn backend.app.main:app --reload --port 8000
# Dashboard: http://127.0.0.1:8000/dashboard
# Swagger: http://127.0.0.1:8000/docs

docker compose build
docker compose up
# API container: 0.0.0.0:8000
# Streamlit container: 0.0.0.0:8501

curl http://127.0.0.1:8000/health
# {"status":"ok","app":"EvalForge API","version":"0.2.0","environment":"local"}
```